

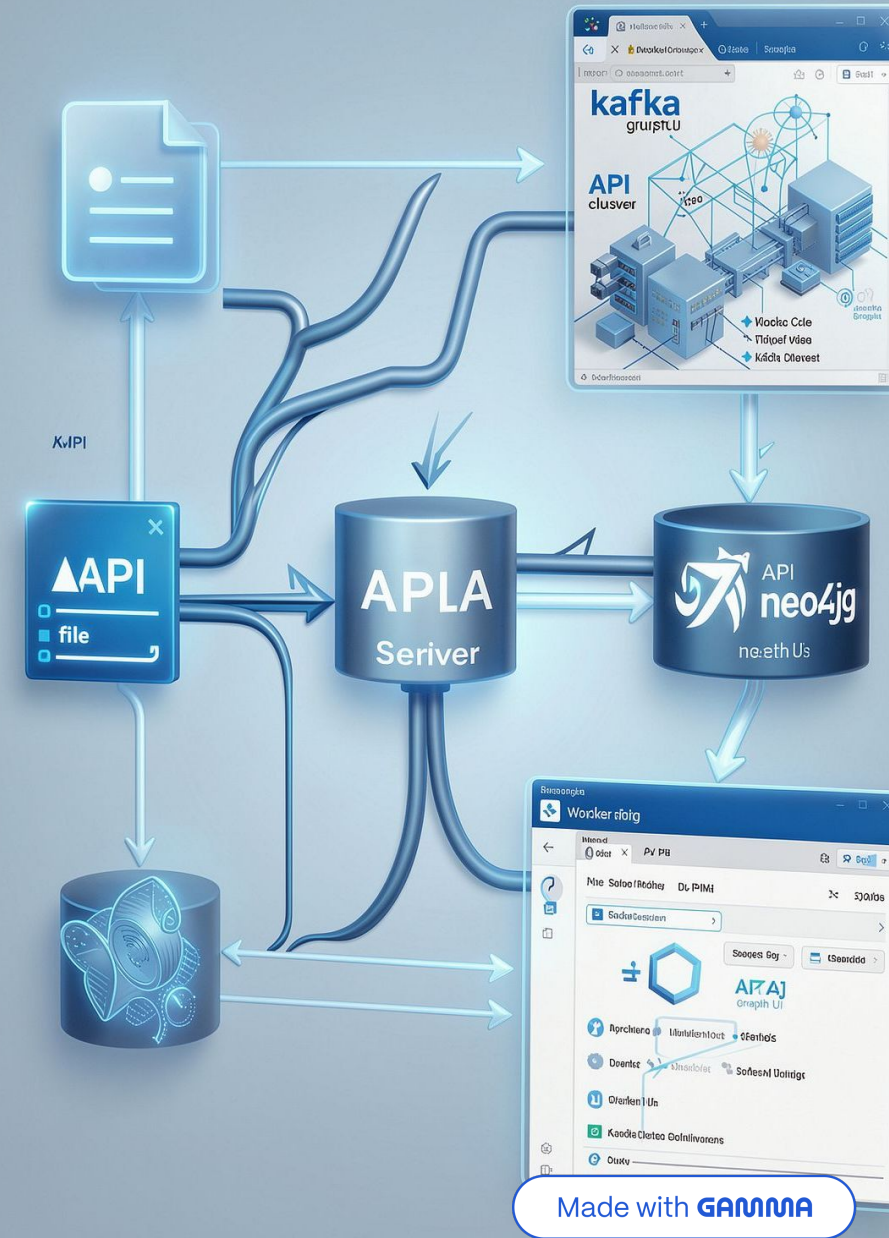
GraphFlow AI — CSV → Graph Ingestion Pipeline

End-to-end architecture for reliably ingesting CSV files, streaming rows, constructing a Neo4j graph, and exposing a grounded natural-language interface. For backend and data engineers evaluating design, reliability, and operational trade-offs.

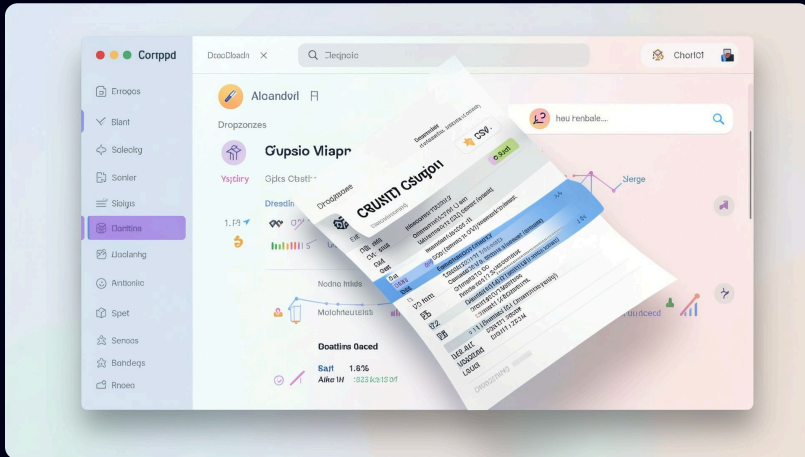


High-level Flow

Five core stages: 1) CSV upload & validation, 2) FastAPI ingestion, 3) Kafka streaming, 4) Loader worker writes to Neo4j, 5) Live UI + grounded chatbot. Decoupling via Kafka ensures uploads never block on DB writes.



Step 1 — Upload & Deterministic Tracking



What happens on POST /ingest

- FastAPI parses CSV, validates header consistency and per-row schema.
- Compute SHA-256 fingerprint for the file content → deterministic dataset_id.
- Generate job_id (short hex, e.g., daae085d) and persist a tracking job (status: queued).
- Respond immediately (202 Accepted) so client does not block on downstream writes.

☐ SHA-256 enables idempotency and simple duplicate-detection across re-uploads.

Step 2 → 3 – Row Streaming & Graph Construction

FastAPI streams each CSV row as a JSON message to Kafka (topic: csv-rows). Messages include dataset_id, row_index, and row data. A dedicated Loader Worker consumes the topic, transforms messages to Cypher MERGE calls, and writes nodes and relationships to Neo4j using the Bolt driver.



Message payload

```
{ "dataset_id": "", "row_index": 42,  
  "data": { "Name": "Arun",  
            "City": "Chennai", ... } }
```

Neo4j model

```
(:Dataset {id}) -[:HAS_ROW]-> (:Row  
  {dataset_id, row_index, ...})
```

Idempotency

Use MERGE on dataset_id and unique properties so re-uploads update not duplicate.

Operational Patterns & Reliability



Decoupling

Kafa buffers bursts and separates ingestion latency from DB throughput—upload latency remains low even for large files.



Exactly-once / Idempotency

SHA-256 dataset_id + MERGE patterns prevent duplicates. Worker is responsible for safe retry semantics.



Monitoring & Backpressure

Track Kafka consumer lag, Neo4j write latency, and job-level progress. Use retention and DLQ for poisoned messages.



Security

Validate inputs, sanitize fields, restrict Bolt user to read/write patterns, and disallow arbitrary query execution from ingestion path.

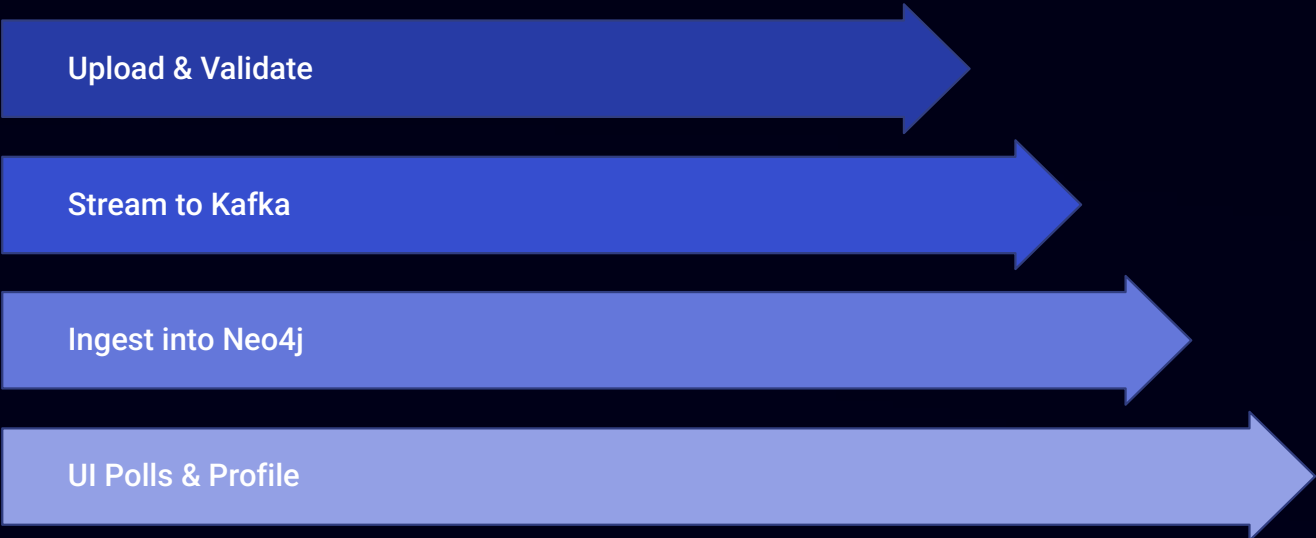
Step 4 → 5 – Live UI & Grounded Chatbot



Chat pipeline (POST /chat)

- Intent & schema detection: fetch active columns from Neo4j schema.
- Cypher codegen: generate safe, read-only queries (MATCH / RETURN) — example: `MATCH (r:Row) WHERE toLower(toString(r.Name)) CONTAINS toLower('arun') RETURN properties(r) LIMIT 5`.
- Security Shield: reject mutating keywords (CREATE, DELETE, DROP). Sanitize inputs and parameterize queries.
- Grounding verification: return "Grounded: False" and refuse to hallucinate when a requested field is absent.

❏ Result formatting: raw Cypher results are converted into concise natural-language answers (e.g., "The City for Arun is Chennai").



Frontend polls GET /status?job_id=... every 500ms to update a stepper and a progress bar (0% → 100%). Dataset preview shows row count, columns, missing-cell analysis, and first five rows.